

SIMP — Sensor IoT Management Protocol

Kamil Janik, Tomasz Kapusta

1. Cel protokołu i zakres

a. Do czego służy protokół?

Protokół SIMP (Sensor IoT Management Protocol) służy do bezpiecznego, dwukierunkowego przesyłania danych pomiędzy sieciowymi urządzeniami pomiarowymi (czujniki temperatury, wilgotności, detektory alarmowe) a centralnym serwerem zbierającym dane. Umożliwia zarówno periodyczne wysyłanie telemetry przez sensory, jak i zdalne wydawanie komend konfiguracyjnych przez serwer.

b. Jakie problemy rozwiązuje?

Protokół rozwiązuje następujące problemy:

- Jak zebrać dane z wielu energooszczędnych urządzeń pracujących na słabym łączu radiowym?
- Jak zapewnić, że dane pochodzą od uwierzytelnionego urządzenia, a nie od intruza?
- Jak obsłużyć chwilowe zerwania połączenia bez utraty danych historycznych?
- Jak minimalizować narzut nagłwkowy, aby oszczędzać baterię urządzenia?

c. Model komunikacji

Protokół działa w modelu wielu-do-jednego: wiele sensorów (klienty, strona aktywna) łączy się z jednym serwerem zbierającym dane (strona pasywna). Sensor jest inicjatorem każdego połączenia TCP; serwer wyłącznie nasłuchuje i odpowiada.

Wzorzec	Kierunek	Przykład
Publish (telemetry)	Sensor -> Serwer	Periodyczne odczyty temperatury
Event-driven (alert)	Sensor -> Serwer	Przekraczanie progu alarmowego
Request-response (komenda)	Serwer -> Sensor	Zmiana interwału pomiarowego
Keep-alive (ping)	Dwukierunkowy	Weryfikacja żywotności połączenia

2. Założenia techniczne

a. Warstwa transportowa

Transport: **TCP + TLS 1.3**. TCP gwarantuje kolejność i dostarczenie segmentów. TLS 1.3 (realizowane przez bibliotekę OpenSSL lub mbedTLS po stronie urządzenia) zapewnia poufność i integralność całego kanału komunikacyjnego.

Serwer nasłuchuje domyślnie na porcie 8883 (analogicznie do MQTT over TLS).

Certyfikat serwera podpisany przez lokalne CA — sensor weryfikuje fingerprint przy pierwszym połączeniu.

b. Kodowanie wiadomości

Wiadomości są kodowane binarnie w formacie sieciowym (big-endian). Wybór kodowania binarnego zamiast tekstowego wynika z wymagań energetycznych: stały narzut nagłówka wynosi jedynie **15 bajtów**, co przy częstej transmisji telemetry z urządzeń bateryjnych przekłada się na istotne oszczędności energii.

c. Odpowiedzialności warstw

Warstwa	Za co odpowiada
TCP	Kolejność, retransmisja bitów, flow control
TSL	Poufność, integralność, uwierzytelnienie serwera
SIMP	Typy wiadomości, uwierzytelnienie sensora, session token, kody błędów, timeouty sesji

3. Struktura komunikatów

a. Format ramki

Każda wiadomość SIMP składa się z nagłówka o stałym rozmiarze **15 bajtów** oraz opcjonalnego pola payload (0–512 bajtów). Nagłówek zawiera wszystkie informacje potrzebne do identyfikacji wiadomości, sesji i jej długości.

Pole	Rozmiar	Opis
version	1 bajt	Wersja protokołu. Aktualnie: 0x01
type	1 bajt	Typ wiadomości
flags	1 bajt	Flagi bitowe: bit 0 = ACK_REQUIRED, bity 1–7 = zarezerwowane (0x00)
session_token	4 bajty	Token sesji wydany przez serwer po AUTH_OK. Przed autoryzacją = 0x00000000
payload_len	2 bajty	Długość pola payload w bajtach (0–512). Big-Endian.
payload	0–512 B	Dane zależne od typu wiadomości

b. Typy wiadomości

Kod	Nazwa	Kierunek	ACK wymagany	Opis
0x01	HELLO	S→Srv	Nie	Inicjacja połączenia, przekazanie device_id
0x02	AUTH	S→Srv	Nie	Przekazanie hasła (pre-shared key)

0x03	AUTH_OK	Srv→S	Nie	Serwer wydaje session_token (4 B w payload)
0x04	AUTH_FAIL	Srv→S	Nie	Błąd uwierzytelnienia, połączenie zostaje zamknięte
0x05	TELEMETRY	S→Srv	Nie	Dane pomiarowe (temp., wilg., stan czujnika)
0x06	ALERT	S→Srv	Tak	Przekroczenie progu alarmowego
0x07	COMMAND	Srv→S	Tak	Komenda dla urządzenia (np. zmień interwał)
0x08	ACK	Obukier.	Nie	Potwierdzenie odebrania wiadomości z flagą ACK_REQUIRED
0x09	PING	Obukier.	Nie	Weryfikacja żywotności połączenia
0x0A	PONG	Obukier.	Nie	Odpowiedź na PING
0x0B	ERROR	Obukier.	Nie	Komunikat błędu z kodem w payload
0x0C	BYE	Obukier.	Nie	Zamknięcie sesji

c. Zawartość payloadu wg typu

Typ	Pola payload (Big-Endian)
HELLO	device_id: 8 bajtów
AUTH	password_len: 1 bajt + password: 1–32 bajty (UTF-8)
AUTH_OK	session_token: 4 bajty
AUTH_FAIL	error_code: 1 bajt
TELEMETRY	sensor_type: 1 B + timestamp: 4 B (Unix) + value: 4 B (float)
ALERT	alert_code: 1 B + sensor_type: 1 B + value: 4 B (float)
COMMAND	cmd_id: 1 B + cmd_seq: 2 B + param_len: 1 B + param: 0–32 B
ACK	ack_seq: 2 bajty (seq numer potwierdzanej komendy/alertu)
PING	(brak — payload_len = 0)
PONG	(brak — payload_len = 0)
ERROR	error_code: 1 B + msg_len: 1 B + msg: 0–64 B (UTF-8)

BYE	(brak — payload_len = 0)
-----	--------------------------

d. Walidacja - co jest błędem formatu?

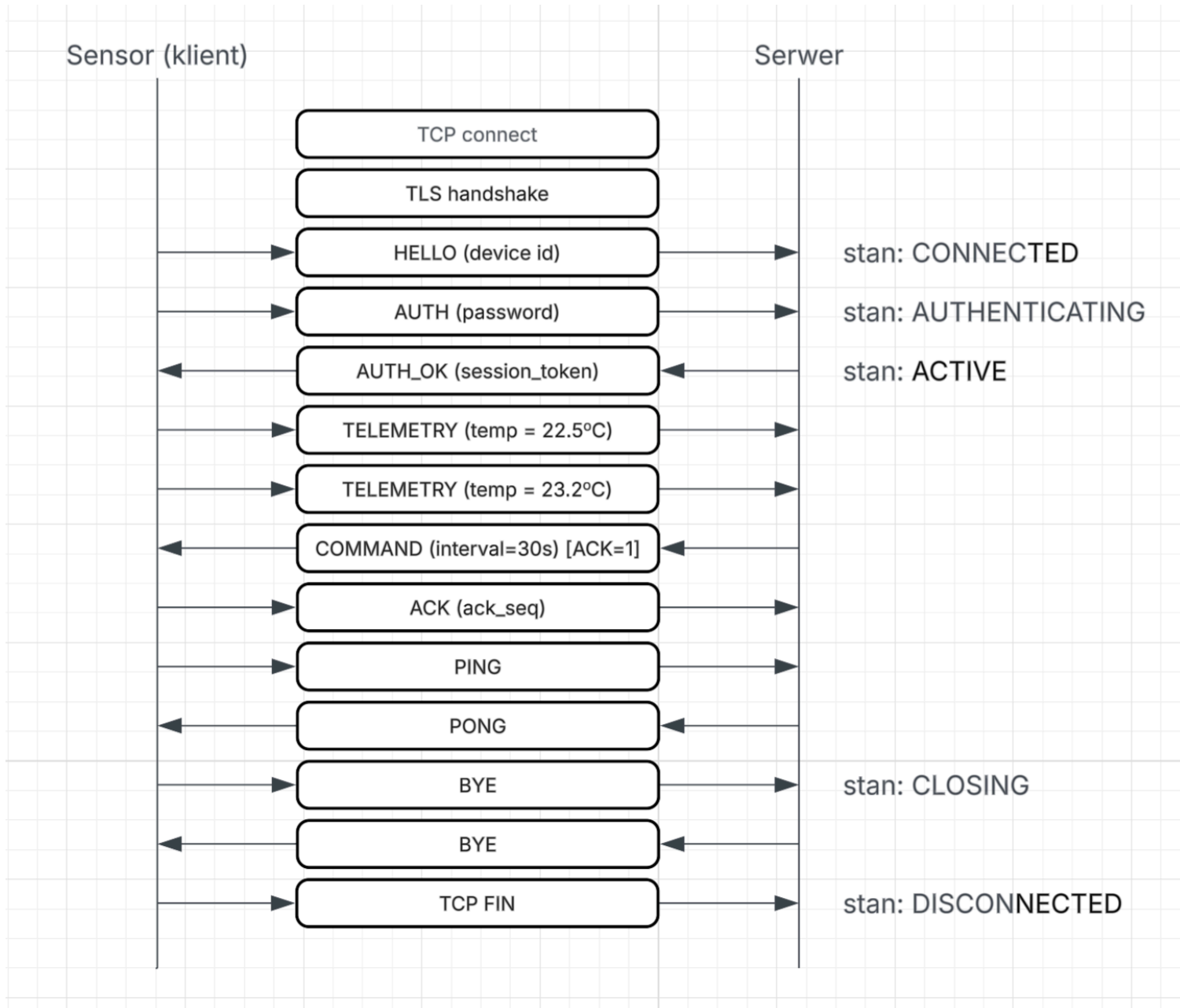
- version $\neq$ 0x01 — nieobsługiwana wersja (ERROR 0x03)
- type spoza zakresu 0x01–0x0C — nieznany typ (ERROR 0x01)
- payload_len > 512 — przekroczenie limitu (ERROR 0x07, połączenie zamykane)
- payload_len niezgodne z faktyczną długością odebranych bajtów — błąd transmisji (ERROR 0x01)
- session_token $\neq$ 0x00000000 dla HELLO/AUTH — protokół naruszony (ERROR 0x02)
- session_token nieznany serwerowi w fazie ACTIVE — nieautoryzowany dostęp (ERROR 0x02)
- Wiadomość TELEMETRY/ALERT/COMMAND przed AUTH_OK — zły stan (ERROR 0x05)

4. Model stanów i przebieg komunikacji

a. Stany sesji

Stan	Opis
DISCONNECTED	Brak połączenia TCP
CONNECTED	TCP nawiązane, TLS handshake
AUTHENTICATING	Serwer weryfikuje device_id + hasło (serwer przetwarza)
ACTIVE	Sesja autoryzowana, wymiana danych
CLOSING	Zainicjowano zamknięcie

b. Diagram sekwencji - nawiązanie i zamknięcie sesji



c. Reguły timeoutów i retransmisji

Sytuacja	Timeout	Zachowanie
Brak AUTH po HELLO	10 s	Serwer wysyła ERROR 0x04, zamyka połączenie
Brak ACK dla ALERT/COMMAND	10 s	Strona wysyłająca ponawia max 3 razy, potem ERROR 0x04
Brak PING/TELEMETRY od sensora	60 s	Serwer wysyła ERROR 0x04, zamyka sesję

Sensor nie otrzymuje PONG	5 s	Sensor ponawia PING raz, potem zamyka TCP i ponawia połączenie
Reconnect po utracie połączenia	5–30 s	Sensor stosuje exponential backoff: 5, 10, 20, 30 s (max)

5. Bezpieczeństwo

a. Poufność

Cały ruch SIMP jest szyfrowany przez TLS 1.3. Po nawiązaniu połączenia TCP sensor inicjuje handshake TLS. Serwer przedstawia certyfikat X.509 podpisany przez lokalne CA (Certificate Authority) projektu. Sensor weryfikuje certyfikat serwera, porównując jego fingerprint SHA-256 z wartością zapisaną w pamięci flash urządzenia podczas konfiguracji (model trust-on-first-use lub provisioning). TLS zapewnia również forward secrecy dzięki użyciu ephemeral Diffie-Hellman.

b. Uwierzytelnienie urządzenia

Po nawiązaniu sesji TLS sensor przesyła sekwencję HELLO -> AUTH:

- Payload HELLO: zawiera device_id (8 bajtów, unikalne ID sprzętowe)
- Payload AUTH: zawiera password_len (1 bajt) oraz password (1-32 bajty, pre-shared key zapisany w flash urządzenia).

Serwer sprawdza, czy para device_id i password istnieje w bazie danych. Jeśli tak, generuje losowy, 4-bajtowy session_token i odsyła go w wiadomości AUTH_OK. W przypadku błędu serwer wysyła AUTH_FAIL i zamyka połączenie. Hasło (pre-shared key) musi być przechowywane w chronionym obszarze pamięci flash urządzenia.

c. Autoryzacja sesji – session_token

Po pomyślnej autoryzacji serwer generuje kryptograficznie losowy session_token (4 bajty, 32 bity entropii). Token jest dołączany do każdej kolejnej ramki w polu session_token nagłówka. Serwer odrzuca ramki z nieznanym lub nieprawidłowym tokenem.

Właściwość	Wartość
Czas życia tokenu	Czas trwania sesji TCP (do BYE lub timeout 60 s)
Generowanie	Serwer: crypto-random (np. os.urandom lub OpenSSL RAND_bytes)
Unieważnianie	Automatyczne przy zamknięciu sesji TCP
Wielokrotne sesje	Każde połączenie TCP otrzymuje nowy, unikatowy token

d. Integralność

Integralność danych jest zapewniana przez warstwę TLS (TLS Record Protocol oblicza kod MAC dla każdego rekordu). Protokół SIMP nie dodaje własnych sum kontrolnych, aby uniknąć zbędnej redundancji przy aktywnym TLS.

e. Ochrona przed replay

TLS 1.3 chroni przed atakami typu replay na poziomie rekordów. Na poziomie SIMP dodatkową ochronę zapewnia krótki czas życia session_token (ważny tylko dla jednej sesji TCP). Dodatkowo pole cmd_seq (2 bajty) w wiadomościach COMMAND chroni przed powtórzeniem komendy w ramach tej samej sesji.

f. Model zagrożeń

Zagrożenie	Wektor ataku	Zabezpieczenie SIMP
Podśluch transmisji	Atakujący nasłuchuje na łączu radiowym	Szyfrowanie całego kanału przez TLS 1.3
Podszywanie się pod serwer	Fałszywy serwer próbuje przejąć dane	Sensor weryfikuje fingerprint certyfikatu serwera
Podszywanie się pod sensor	Atakujący wysyła dane z obcym device_id	Wymagane device_id + pre-shared password; bez hasła następuje AUTH_FAIL
Replay attack	Odtworzenie przechwyconych ramek	TLS anti-replay + krótki TTL tokenu sesji + cmd_seq
Brute-force hasła	Próby zgadnięcia pre-shared key	Rate-limit: 3 próby AUTH na 60 s z danego IP; blokada po przekroczeniu
Przepełnienie bufora (DoS)	Ramki z payload_len > 512	Walidacja nagłówka przed alokacją; zamknięcie sesji przy naruszeniu
Flooding nowych sesji (DoS)	Masowe połączenia TCP bez AUTH	Timeout 10 s na AUTH; limit sesji na IP

6. Obsługa błędów i awarii połączenia

a. Kody błędów

Kod (hex)	Nazwa	Opis	Działanie po błędzie
0x01	FORMAT_ERROR	Błędna struktura ramki lub nieznany typ	Serwer wysyła ERROR, zamyka sesję
0x02	AUTH_INVALID	Nieprawidłowe device_id lub hasło	Serwer wysyła AUTH_FAIL, zamyka TCP
0x03	VERSION_MISMATCH	Nieobsługiwana wersja protokołu	ERROR + zamknięcie sesji
0x04	TIMEOUT	Upłynął timeout sesji lub oczekiwania	ERROR + zamknięcie sesji
0x05	STATE_ERROR	Wiadomość niedozwolona w bieżącym stanie	ERROR (sesja może pozostać)
0x06	INTERNAL_ERROR	Błąd wewnętrzny serwera	ERROR (sesja może pozostać)
0x07	PAYLOAD_TOO_LARGE	payload_len > 512 bajtów	ERROR + natychmiastowe zamknięcie sesji
0x08	RATE_LIMITED	Przekroczono limit prób AUTH lub połączeń	ERROR + zamknięcie TCP + cooldown

b. Zachowanie po zerwaniu połączenia

Sensor wykrywa zerwanie połączenia poprzez brak odpowiedzi PONG (w ciągu 5 s od PING), błąd zapisu do socketu TCP lub upływanie czasu oczekiwania na AUTH_OK / ACK. Po wykryciu awarii sensor:

- Zamyka gniazdo TCP.
- Buforuje lokalnie do 100 rekordów telemetry (ring buffer w pamięci flash).
- Stosuje exponential backoff: 5s->10s->20s->30s (maksymalnie) przed kolejną próbą.
- Po połączeniu wykonuje pełną sekwencję TLS + HELLO + AUTH (uzyskuje nowy session_token).
- Wysyła zbuforowane dane telemetryczne z historycznymi znacznikami czasu (timestamp).

c. Duplikaty i niekompletne wiadomości

TCP gwarantuje brak duplikatów na poziomie strumienia, jednak przy ponownym łączeniu sensor może ponownie wysłać ALERT lub COMMAND, które nie zostały potwierdzone. Pole cmd_seq w wiadomości COMMAND pozwala serwerowi

zignorować duplikat, odpowiadając jedynie ACK, by zatrzymać retransmisję ze strony sensora.

d. Ochrona przed nadużyciami

Mechanizm	Wartość domyślna
Maks. rozmiar ramki	531 bajtów (nagłówek + payload maks. 512 B + padding)
Maks. próby AUTH na IP / 60 s	3 (przekroczenie: błąd 0x08, ban IP na 300 s)
Maks. otwartych sesji na IP	5
Timeout w stanie CONNECTED (bez AUTH)	10s
Timeout sesji ACTIVE (bez ruchu)	60s
Maks. komend COMMAND bez ACK (kolejka)	8 (przekroczenie: ERROR + zamknięcie)

7. Przykładowe scenariusze komunikacji

a. Scenariusz poprawny – pełna sesja z telemetrią i komendą

1. SENSOR: [TCP connect: port 8883].
2. SENSOR: [TLS 1.3 handshake – certyfikat OK].
3. SENSOR -> SRV: HELLO [v=1, type=0x01, device_id=0xDEADBEEFCAFEBABE].
4. SENSOR -> SRV: AUTH [type=0x02, password='secret_key'].
5. SRV: [weryfikacja hasła pomyślna].
6. SRV -> SENSOR: AUTH_OK [type=0x03, token=0xA1B2C3D4].
7. SENSOR -> SRV: TELEMETRY [sess=0xA1B2C3D4, type=TEMP, val=22.5].
8. SENSOR -> SRV: ALERT [flags=ACK_REQUIRED, sess=0xA1B2C3D4, code=HIGH_TEMP, val=41.3].
9. SRV -> SENSOR: ACK [sess=0xA1B2C3D4, ack_seq=0x0001].
10. SRV -> SENSOR: COMMAND [flags=ACK_REQUIRED, cmd_id=SET_INTERVAL, param=30s].
11. SENSOR -> SRV: ACK [sess=0xA1B2C3D4, ack_seq=0x0001].
12. SENSOR -> SRV: BYE [sess=0xA1B2C3D4].
13. SENSOR: [TCP close].

b. Scenariusz błędu – nieautoryzowany dostęp

1. ATAKUJĄCY: [TCP connect + TLS handshake].
2. ATAKUJĄCY -> SRV: HELLO [device_id=...].
3. ATAKUJĄCY -> SRV: AUTH [password='wrong_password'].
4. SRV: [hash mismatch].
5. SRV -> ATAKUJĄCY: AUTH_FAIL [error_code=0x02].
6. SRV: [TCP close]. (*Po 3 próbach w ciągu 60 s następuje rate-limit*).
7. ATAKUJĄCY: [kolejna próba TCP connect].
8. SRV -> ATAKUJĄCY: ERROR [0x08 RATE_LIMITED].
9. SRV: [TCP close; IP zablokowane na 300 s].

c. Scenariusz błędu – zerwanie połączenia w trakcie sesji

1. Sesja aktywna, przesłano 5 ramek TELEMETRY.
2. ZERWANIE ŁĄCZA RADIOWEGO.
3. SENSOR: brak PONG -> wykrycie zerwania, buforowanie danych lokalnie.
4. SENSOR: czeka 5 s, potem 10 s (backoff).
5. SRV: 60 s bez ruchu -> błąd TIMEOUT, zamknięcie sesji.
6. SENSOR: nowe połączenie TCP + TLS + HELLO + AUTH.
7. SRV -> SENSOR: AUTH_OK [nowy token=0xF9E8D7C6].
8. SENSOR -> SRV: przesyłanie danych historycznych z bufora, następnie danych bieżących.

d. Scenariusz błędu – błędny format ramki

1. SENSOR: przesyła ramkę z type=0xFF (nieznany typ).
2. SRV: walidacja nagłówka wykrywa błąd.
3. SRV -> SENSOR: ERROR [0x01 FORMAT_ERROR, msg='Unknown message type: 0xFF'].
4. SRV: natychmiastowe zamknięcie połączenia TCP.